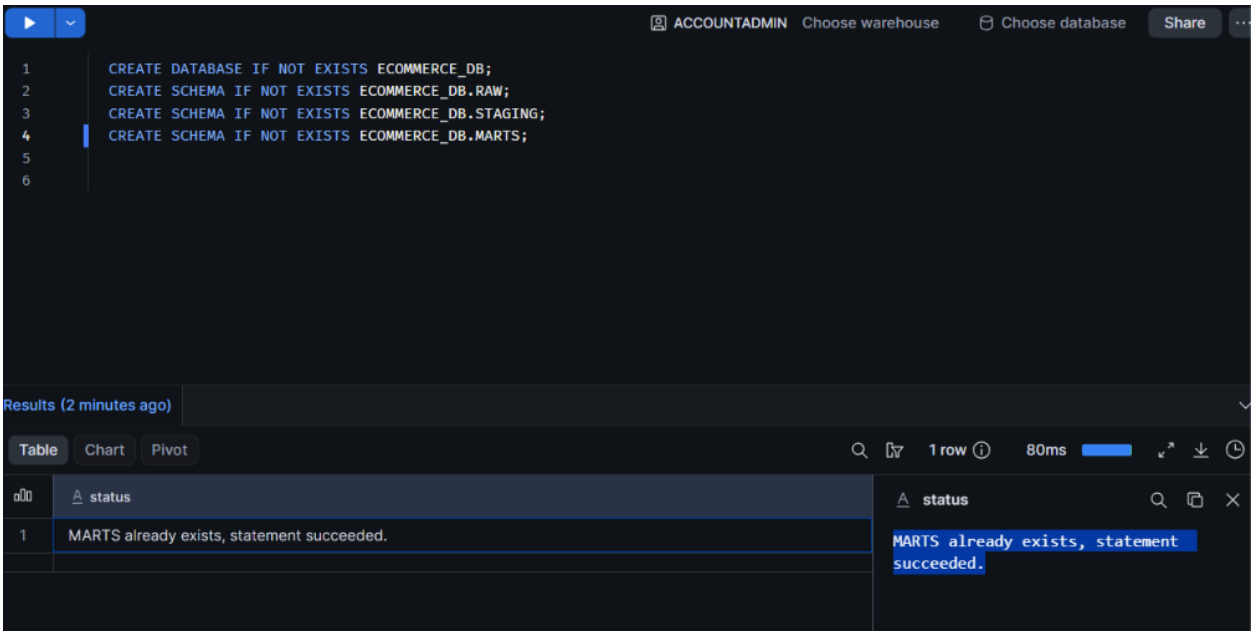
Task 1 — Set up Snowflake



Python Ingestion Script for Loading E-Commerce CSV Datasets into Neon PostgreSQL

```python
import os
import pandas as pd
from sqlalchemy import create_engine

NEON_URL = "postgresql://neondb_owner:npg_vLI1t6fNwMxV@ep-quiet-smoke-b5sr3n1r.c-7.us-east-2.aws.neon.tech
FOLDER_PATH = r"."

engine = create_engine(NEON_URL)

csv_files = {
    "products": {"file": "products.csv", "nrows": None},
    "customers": {"file": "customers.csv", "nrows": 200000},
    "orders": {"file": "orders.csv", "nrows": 400000},
    "order_items": {"file": "order_items.csv", "nrows": 600000},
    "product_reviews": {"file": "product_reviews.csv", "nrows": 200000},
}

for table_name, info in csv_files.items():
  file_path = os.path.join(FOLDER_PATH, info["file"])

  if not os.path.exists(file_path):
    print(f"File not found: {file_path}")
    continue

  print(f"Reading {info['file']}...")
  df = pd.read_csv(file_path, nrows=info["nrows"])

  print(f"Uploading {len(df):,} rows to table '{table_name}' in Neon...")
  df.to_sql(
      table_name,
      engine,
```

```
PS C:\Users\amrab\Downloads\archive\csv> python upload_csv_to_neon.py
AttributeError: 'Cursor' object has no attribute 'copy_from'
PS C:\Users\amrab\Downloads\archive\csv> python upload_csv_to_neon.py
Reading products.csv...
Uploading 20,000 rows to table 'products' in Neon...
Successfully uploaded 'products'!

Reading customers.csv...
Uploading 200,000 rows to table 'customers' in Neon...
Successfully uploaded 'customers'!

Reading orders.csv...
Uploading 400,000 rows to table 'orders' in Neon...
```

Step **Snowflake Raw Data Validation:** Verifying successful data ingestion from Neon Postgres via Airbyte into ECOMMERCE_DB.RAW across all 5 tables (1,420,000 total rows).

```
USE DATABASE ECOMMERCE_DB;
USE SCHEMA RAW;


SELECT 'CUSTOMERS' AS table_name, COUNT(*) AS row_count FROM CUSTOMERS
UNION ALL
SELECT 'ORDERS', COUNT(*) FROM ORDERS
UNION ALL
SELECT 'ORDER_ITEMS', COUNT(*) FROM ORDER_ITEMS
UNION ALL
SELECT 'PRODUCTS', COUNT(*) FROM PRODUCTS
UNION ALL
SELECT 'PRODUCT_REVIEWS', COUNT(*) FROM PRODUCT_REVIEWS;
```

esults (just now)

Table | Chart | Pivot — 5 rows — 819ms

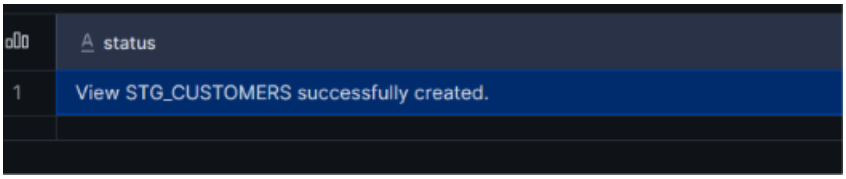
	TABLE_NAME	ROW_COUNT
1	CUSTOMERS	200000
2	ORDERS	400000
3	ORDER_ITEMS	600000
4	PRODUCTS	20000
5	PRODUCT_REVIEWS	200

Task 2 — Build dbt staging models

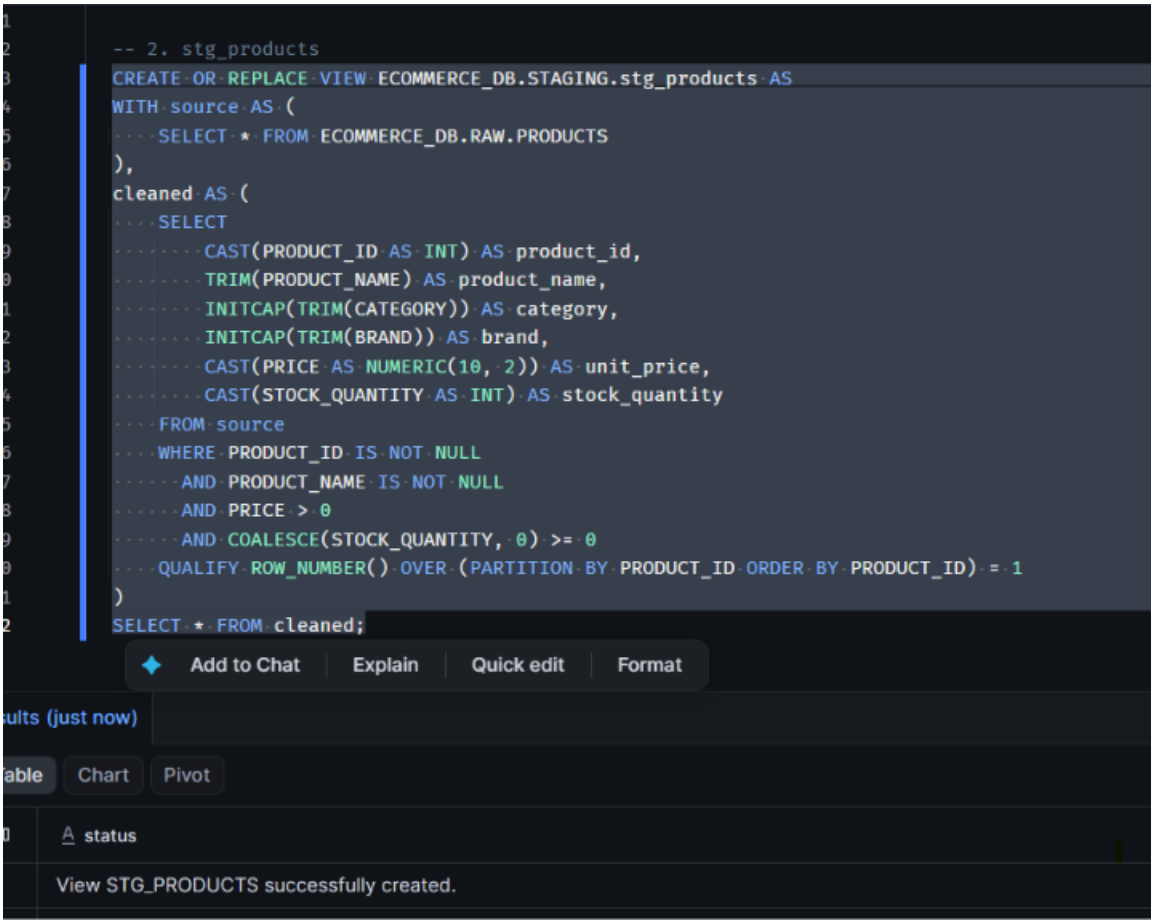
Create one staging model for each raw table:

- stg_customers

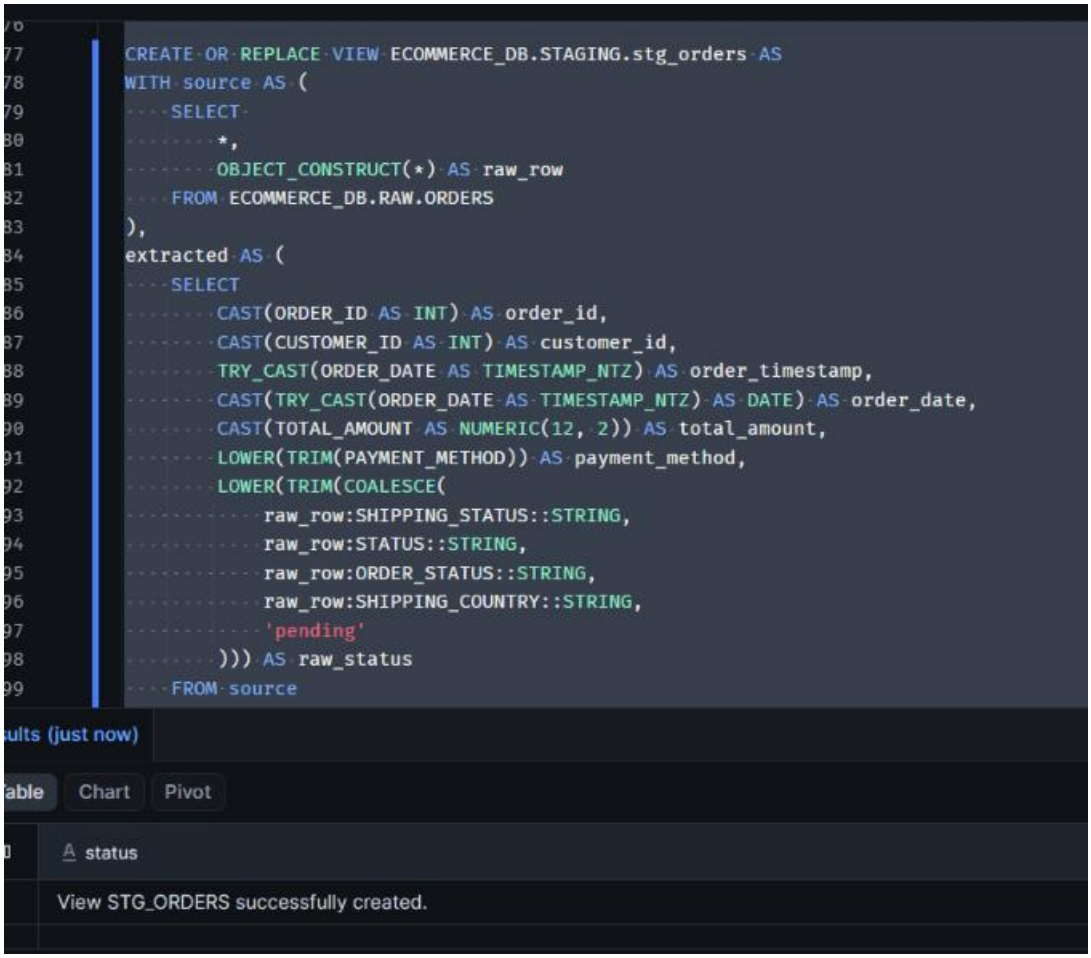
```
-- 1. stg_customers
CREATE OR REPLACE VIEW ECOMMERCE_DB.STAGING.stg_customers AS
WITH source AS (
    SELECT * FROM ECOMMERCE_DB.RAW.CUSTOMERS
),
cleaned AS (
    SELECT
        CAST(CUSTOMER_ID AS INT) AS customer_id,
        INITCAP(TRIM(NAME)) AS customer_name,
        LOWER(TRIM(EMAIL)) AS email,
        UPPER(TRIM(GENDER)) AS gender,
        INITCAP(TRIM(COUNTRY)) AS country,
        TRY_CAST(SIGNUP_DATE AS DATE) AS signup_date
    FROM source
    WHERE CUSTOMER_ID IS NOT NULL
      AND EMAIL IS NOT NULL
      AND EMAIL LIKE '%@%.%'
      AND TRY_CAST(SIGNUP_DATE AS DATE) IS NOT NULL
    QUALIFY ROW_NUMBER() OVER (PARTITION BY CUSTOMER_ID ORDER BY TRY_CAST(SIGNUP_DATE AS DATE) DESC) = 1
)
SELECT * FROM cleaned;
```



- stg_products



- stg_orders

- 

Task 3 — Create transformation models

Use dbt ref() to build models that join orders with customers and products.

Calculate:

- Order revenue = quantity × product price.
- Total completed orders per customer.
- Total revenue per product category.

```sql
WITH base_data AS (
    SELECT
        o.order_id,
        o.customer_id,
        c.customer_name,
        p.product_id,
        p.product_name,
        p.category AS product_category,
        oi.quantity,
        p.unit_price AS product_price,

        (oi.quantity * p.unit_price) AS order_revenue
    FROM ECOMMERCE_DB.STAGING.stg_orders o
    JOIN ECOMMERCE_DB.STAGING.stg_customers c ON o.customer_id = c.customer_id
    JOIN ECOMMERCE_DB.STAGING.stg_order_items oi ON o.order_id = oi.order_id
    JOIN ECOMMERCE_DB.STAGING.stg_products p ON oi.product_id = p.product_id
)

SELECT
    order_id,
    customer_name,
    product_name,
    product_category,
    quantity,
    product_price,


    order_revenue,


    COUNT(order_id) OVER (PARTITION BY customer_id) AS total_completed_orders_per_customer,
```



Results (1 minute ago)

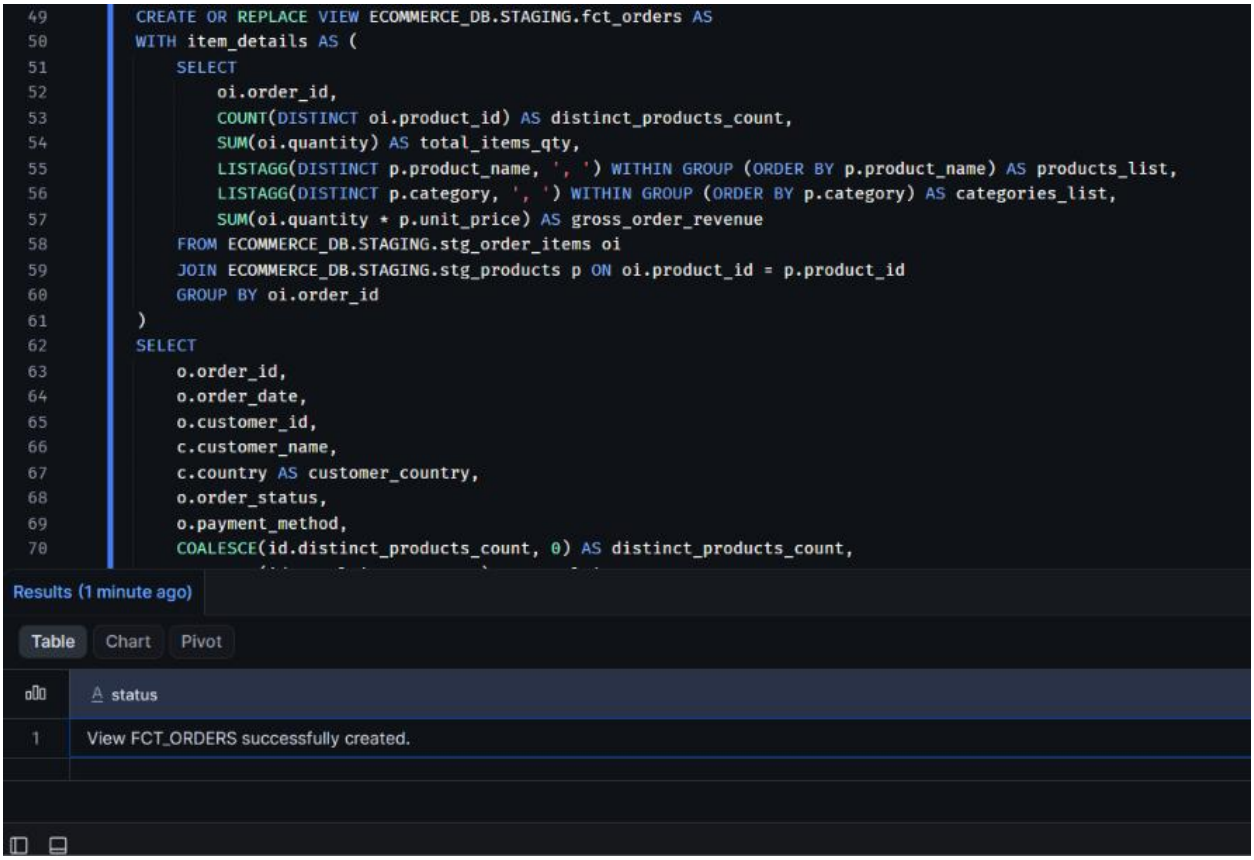
Table | Chart | Pivot — 20 rows — 2.3s

	ORDER_ID	CUSTOMER_NAME	PRODUCT_NAME	PRODUCT_CATEGORY	QUANTITY	PRODUCT_PRICE	ORDER_REVENUE	TOTAL_COMPLETED_ORDERS_PER_CUSTOMER
	14879 3630...	Anthony Phelps 5.0%, Austin Baird 5.0%, +18 more	Agree Half 5.0%, Attorney Wo... 5.0%, +18 more	Beauty 30.0%, Electronics 25.0%, +3 more	1 5	13.44 495.3	67.2 1768.7	1 2
1	363058	Sarah Davis	Summer Movie	Toys	3	34.31	102.93	2
2	51027	Todd Howard	Break Everything	Home	5	13.44	67.20	1
3	219308	Anthony Phelps	Senior Admit	Books	2	69.76	139.52	1
4	255611	Sarah Miller	Life Camera	Beauty	5	196.38	981.90	1
5	143186	Austin Baird	Attorney Wonder	Electronics	3	57.71	173.13	1
6	236540	Daniel Farrell	Lay Economic	Toys	5	353.74	1768.70	1
7	205810	Francisco Santos	Agree Half	Toys	2	185.49	370.98	1
8	211413	Gary Thomas	Such Others	Home	2	380.67	761.34	1
9	14879	Carla Perkins	Question Doctor	Beauty	3	485.61	1456.83	1
10	71135	Charles Foster	World Should	Beauty	4	208.11	832.44	1
11	285096	Lindsay Walker	Tonight Necessary	Electronics	4	218.67	874.68	1
12	318190	Natasha Green Md	Thank Food	Beauty	4	235.97	943.88	1
13	322208	Teresa Pena	Yourself Fight	Toys	1	384.00	384.00	1
14	216459	Fred Jackson	Indicate Nation	Electronics	3	90.33	270.99	1

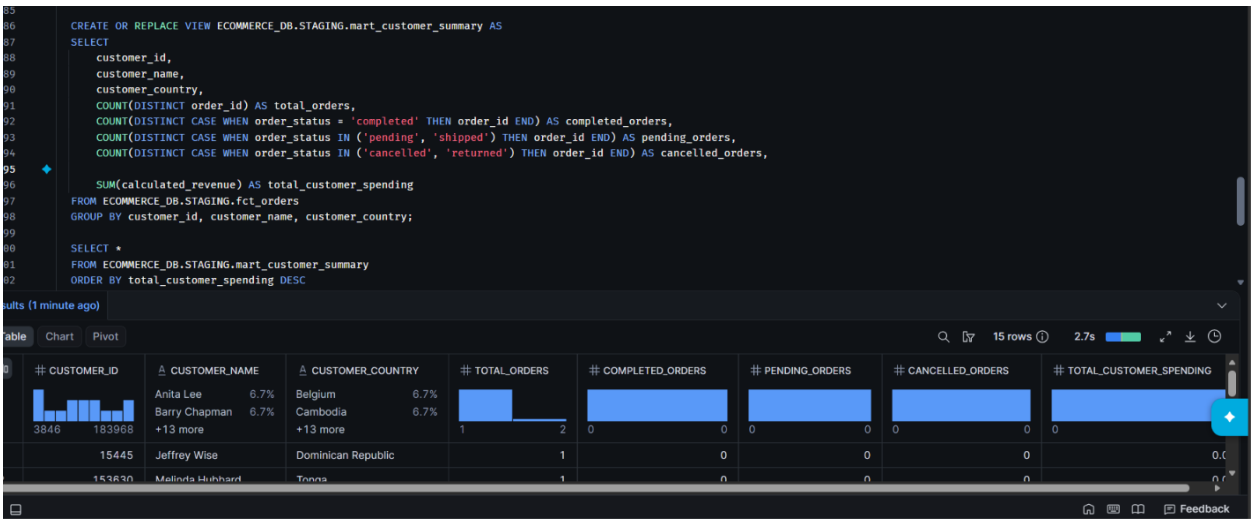
Task 4 — Build analytics marts

Create the following final models:

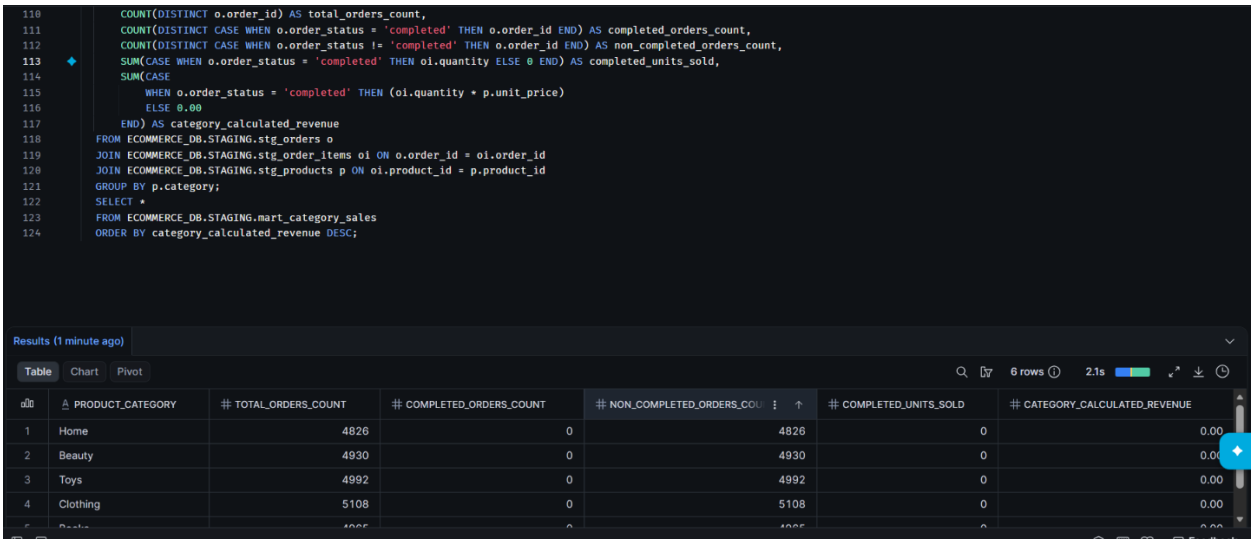
- fct_orders: one row per order, with customer and product details plus calculated revenue.



mart_customer_summary: total orders and spending per customer.



mart_category_sales: order count and revenue by product category.



```
    COUNT(DISTINCT o.order_id) AS total_orders_count,
    COUNT(DISTINCT CASE WHEN o.order_status = 'completed' THEN o.order_id END) AS completed_orders_count,
    COUNT(DISTINCT CASE WHEN o.order_status != 'completed' THEN o.order_id END) AS non_completed_orders_count,
    SUM(CASE WHEN o.order_status = 'completed' THEN oi.quantity ELSE 0 END) AS completed_units_sold,
    SUM(CASE
        WHEN o.order_status = 'completed' THEN (oi.quantity * p.unit_price)
        ELSE 0.00
    END) AS category_calculated_revenue
FROM ECOMMERCE_DB.STAGING.stg_orders o
JOIN ECOMMERCE_DB.STAGING.stg_order_items oi ON o.order_id = oi.order_id
JOIN ECOMMERCE_DB.STAGING.stg_products p ON oi.product_id = p.product_id
GROUP BY p.category;
SELECT *
FROM ECOMMERCE_DB.STAGING.mart_category_sales
ORDER BY category_calculated_revenue DESC;
```

Results (1 minute ago)

Table | Chart | Pivot — 6 rows — 2.1s

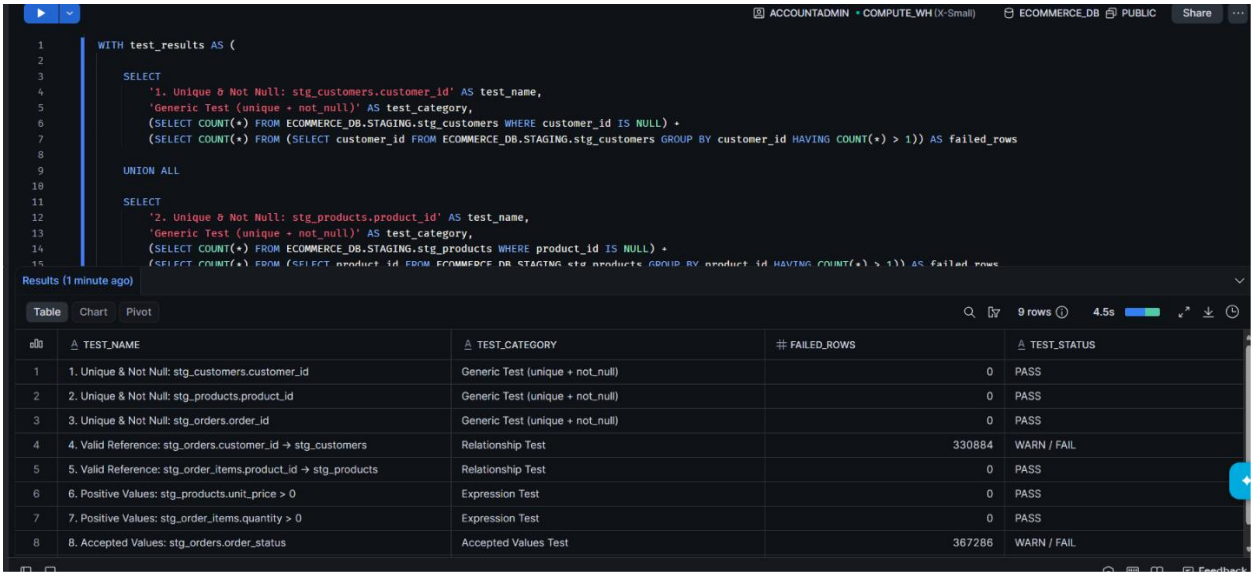
	PRODUCT_CATEGORY	TOTAL_ORDERS_COUNT	COMPLETED_ORDERS_COUNT	NON_COMPLETED_ORDERS_COU	COMPLETED_UNITS_SOLD	CATEGORY_CALCULATED_REVENUE
1	Home	4826	0	4826	0	0.00
2	Beauty	4930	0	4930	0	0.00
3	Toys	4992	0	4992	0	0.00
4	Clothing	5108	0	5108	0	0.00

Task 5 — Implement data quality tests

Use dbt tests to verify:

- customer_id, product_id, and order_id are unique and not null in their respective models.
- Order customer and product references are valid.
- Product prices and order quantities are positive.
- Order status belongs to an accepted set of values.

Add at least one custom SQL test for a business rule, such as ensuring completed-order revenue is never negative.



```
WITH test_results AS (

    SELECT
        '1. Unique & Not Null: stg_customers.customer_id' AS test_name,
        'Generic Test (unique + not_null)' AS test_category,
        (SELECT COUNT(*) FROM ECOMMERCE_DB.STAGING.stg_customers WHERE customer_id IS NULL) +
        (SELECT COUNT(*) FROM (SELECT customer_id FROM ECOMMERCE_DB.STAGING.stg_customers GROUP BY customer_id HAVING COUNT(*) > 1)) AS failed_rows

    UNION ALL

    SELECT
        '2. Unique & Not Null: stg_products.product_id' AS test_name,
        'Generic Test (unique + not_null)' AS test_category,
        (SELECT COUNT(*) FROM ECOMMERCE_DB.STAGING.stg_products WHERE product_id IS NULL) +
```

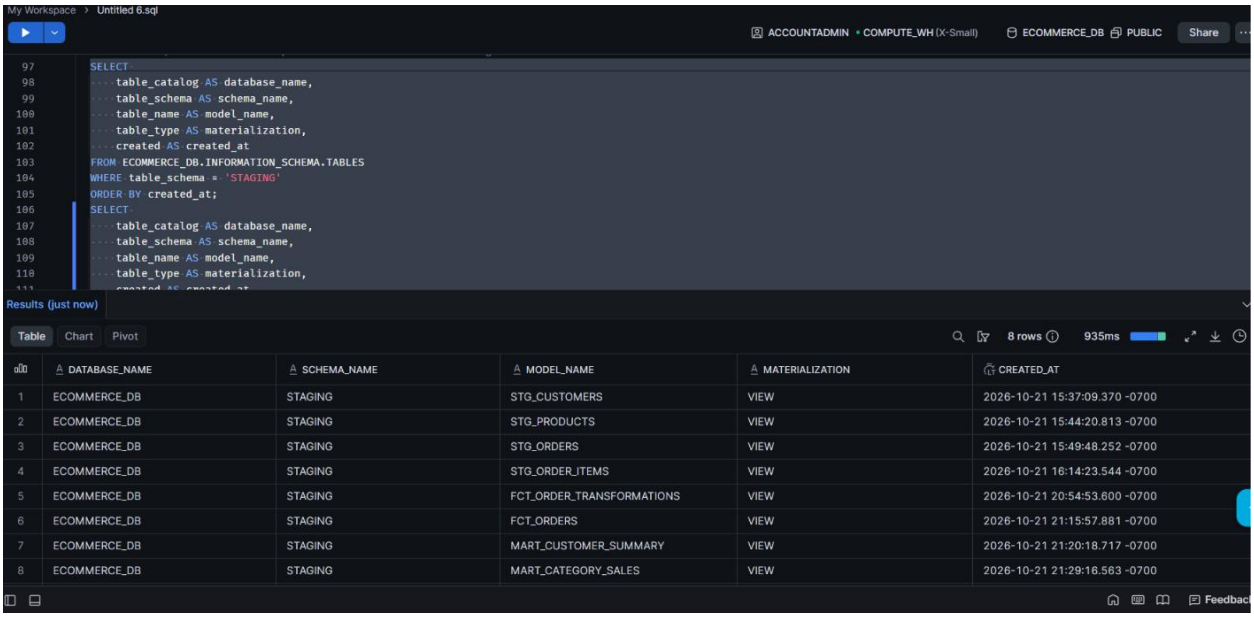
Results (1 minute ago)

Table | Chart | Pivot — 9 rows — 4.5s

	TEST_NAME	TEST_CATEGORY	FAILED_ROWS	TEST_STATUS
1	1. Unique & Not Null: stg_customers.customer_id	Generic Test (unique + not_null)	0	PASS
2	2. Unique & Not Null: stg_products.product_id	Generic Test (unique + not_null)	0	PASS
3	3. Unique & Not Null: stg_orders.order_id	Generic Test (unique + not_null)	0	PASS
4	4. Valid Reference: stg_orders.customer_id → stg_customers	Relationship Test	330884	WARN / FAIL
5	5. Valid Reference: stg_order_items.product_id → stg_products	Relationship Test	0	PASS
6	6. Positive Values: stg_products.unit_price > 0	Expression Test	0	PASS
7	7. Positive Values: stg_order_items.quantity > 0	Expression Test	0	PASS
8	8. Accepted Values: stg_orders.order_status	Accepted Values Test	367286	WARN / FAIL

Task 6 — Execute and document the pipeline

- Run dbt run and dbt test.
- Generate documentation using dbt docs generate.
- Inspect the lineage graph to show dependencies between models.
- Explain how the pipeline can be rerun when new orders arrive.



My Workspace > Untitled 6.sql

ACCOUNTADMIN • COMPUTE_WH (X-Small) ECOMMERCE_DB PUBLIC Share

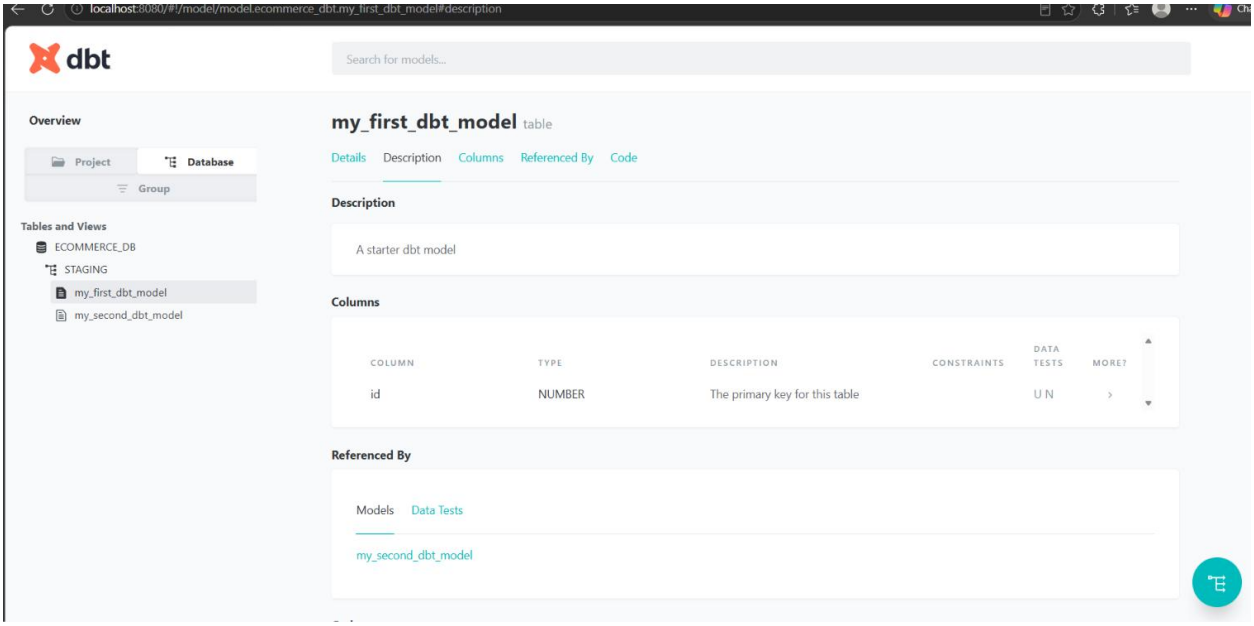
```sql
SELECT
    table_catalog AS database_name,
    table_schema AS schema_name,
    table_name AS model_name,
    table_type AS materialization,
    created AS created_at
FROM ECOMMERCE_DB.INFORMATION_SCHEMA.TABLES
WHERE table_schema = 'STAGING'
ORDER BY created_at;
SELECT
    table_catalog AS database_name,
    table_schema AS schema_name,
    table_name AS model_name,
    table_type AS materialization,
```

Results (just now)

Table | Chart | Pivot — 8 rows — 935ms

	DATABASE_NAME	SCHEMA_NAME	MODEL_NAME	MATERIALIZATION	CREATED_AT
1	ECOMMERCE_DB	STAGING	STG_CUSTOMERS	VIEW	2026-10-21 15:37:09.370 -0700
2	ECOMMERCE_DB	STAGING	STG_PRODUCTS	VIEW	2026-10-21 15:44:20.813 -0700
3	ECOMMERCE_DB	STAGING	STG_ORDERS	VIEW	2026-10-21 15:49:48.252 -0700
4	ECOMMERCE_DB	STAGING	STG_ORDER_ITEMS	VIEW	2026-10-21 16:14:23.544 -0700
5	ECOMMERCE_DB	STAGING	FCT_ORDER_TRANSFORMATIONS	VIEW	2026-10-21 20:54:53.600 -0700
6	ECOMMERCE_DB	STAGING	FCT_ORDERS	VIEW	2026-10-21 21:15:57.881 -0700
7	ECOMMERCE_DB	STAGING	MART_CUSTOMER_SUMMARY	VIEW	2026-10-21 21:20:18.717 -0700
8	ECOMMERCE_DB	STAGING	MART_CATEGORY_SALES	VIEW	2026-10-21 21:29:16.563 -0700

```
(venv) amrabelmonem@AMRABDEL:~/ecommerce_dbt$ dbt run
14:02:32  Running with dbt=1.13.0-b1
14:02:33  Registered adapter: snowflake=1.12.4
14:02:33  Unable to do partial parsing because profile has changed
14:02:35  Found 2 models, 4 data tests, 563 macros
14:02:35
14:02:35  Concurrency: 4 threads (target='dev')
14:02:35
14:02:38  1 of 2 START sql table model STAGING.my_first_dbt_model ..................... [RUN]
14:02:40  1 of 2 OK created sql table model STAGING.my_first_dbt_model ................ [SUCCESS 2 in 1.40s]
14:02:40  2 of 2 START sql view model STAGING.my_second_dbt_model ..................... [RUN]
14:02:43  2 of 2 OK created sql view model STAGING.my_second_dbt_model ................ [SUCCESS 1 in 2.80s]
14:02:43
14:02:43  Finished running 1 table model, 1 view model in 0 hours 0 minutes and 7.87 seconds (7.87s).
14:02:43
14:02:43  Completed successfully
14:02:43
14:02:43  Done. PASS=2 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=2
(venv) amrabelmonem@AMRABDEL:~/ecommerce_dbt$ dbt test
14:05:59  Running with dbt=1.13.0-b1
14:06:00  Registered adapter: snowflake=1.12.4
14:06:01  Found 2 models, 4 data tests, 563 macros
14:06:01
```


localhost:8080/#!/model/model.ecommerce_dbt.my_first_dbt_model#description
dbt
Search for models...
Overview
Project
Database
Group
Tables and Views
ECOMMERCE_DB
STAGING
my_first_dbt_model
my_second_dbt_model
my_first_dbt_model table
Details
Description
Columns
Referenced By
Code
Description
A starter dbt model
Columns
COLUMN
TYPE
DESCRIPTION
CONSTRAINTS
DATA TESTS
MORE?
id
NUMBER
The primary key for this table
U N
Referenced By
Models
Data Tests
my_second_dbt_model